Classificação de Veículos Utilizando Técnicas Tradicionais e Deep Learning

Henrique Zan Grande, João Gabriel Pitol,
Lucas Braga Schuck, Rafael Vargas Serenato

Visão Computacional – PUCPR

2025

Tarefa

O trabalho consiste em um **problema de classificação de imagens**, cujo objetivo é identificar diferentes tipos de veículos.

Essa tarefa é relevante na **Visão Computacional**, com aplicações em **sistemas de tráfego, monitoramento e reconhecimento automático**, permitindo comparar métodos tradicionais e de **deep learning** quanto à precisão e generalização.

Conjunto de Dados

Foi utilizado o **Vehicles Dataset**, um conjunto de imagens públicas disponível no **Kaggle**.

Contendo **5.590 imagens** distribuídas em **7 classes**: *Auto Rickshaws, Bikes, Cars, Motorcycles, Planes, Ships* e *Trains*.

Todas as imagens foram **padronizadas para 224×224 pixels**, com **distribuição balanceada** entre as classes (cerca de **800 imagens por categoria**, exceto “Cars” com 790).

Após o **data augmentation**, o conjunto de treino passou a conter **7.824 imagens**, mantendo o mesmo número de classes e resolução.

Conjunto de Dados

Divisão do Conjunto de Dados

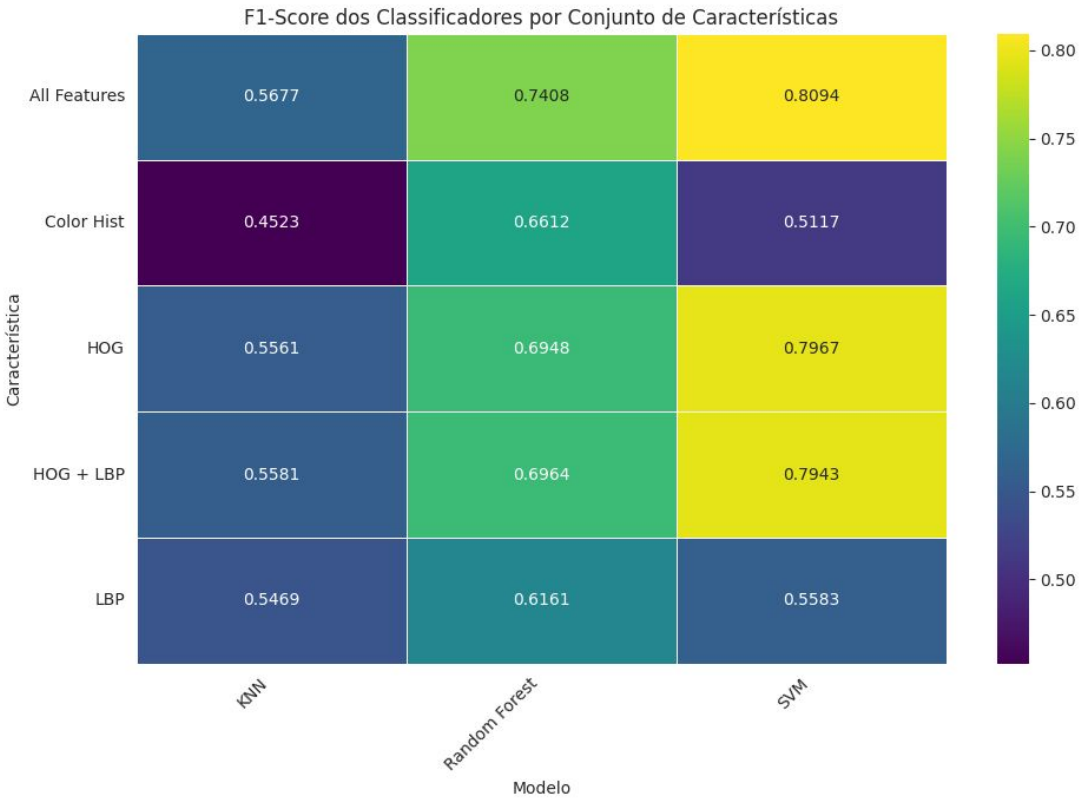
O dataset foi dividido em **70% treino**, **15% validação** e **15% teste**, garantindo **dados suficientes para aprendizado** e uma **avaliação confiável da generalização**. Como o conjunto é **balanceado e de tamanho moderado**, essa divisão assegura um **treinamento eficiente** sem prejudicar a **qualidade da validação e do teste**.

Pré-processamento

Todas as imagens foram **redimensionadas para 224×224 pixels** e convertidas para **RGB**, garantindo compatibilidade com as arquiteturas de rede.
Durante a **extração de características**, aplicaram-se conversões para **HSV** e **escala de cinza** (usadas em histogramas de cor, LBP e HOG).
Também foi feita a **normalização dos pixels para [0,1]** e utilizados **pré-processamentos específicos** das arquiteturas **MobileNetV2** e **EfficientNetB0**.

Abordagem Tradicional (Baseline)

Na abordagem tradicional, a melhor combinação foi obtida com o **descritor HOG** aliado ao **classificador SVM**, alcançando a **maior acurácia e desempenho geral** entre os métodos testados.



Data Augmentation

Foram aplicadas técnicas clássicas de *data augmentation* para aumentar a diversidade e a robustez do conjunto de treino, preservando sempre a coloração original das imagens. As transformações utilizadas incluíram **flip horizontal**, **rotações**, **translações**, **variações leves de escala (zoom in/out)** e **adição de ruído ou desfoque gaussiano**.

O processo foi realizado de forma **offline**, ampliando o dataset em aproximadamente **30%**, resultando em um conjunto aumentado com imagens adicionais geradas a partir das amostras originais, todas mantidas na resolução **224×224**..

Data Augmentation

Deep Learning

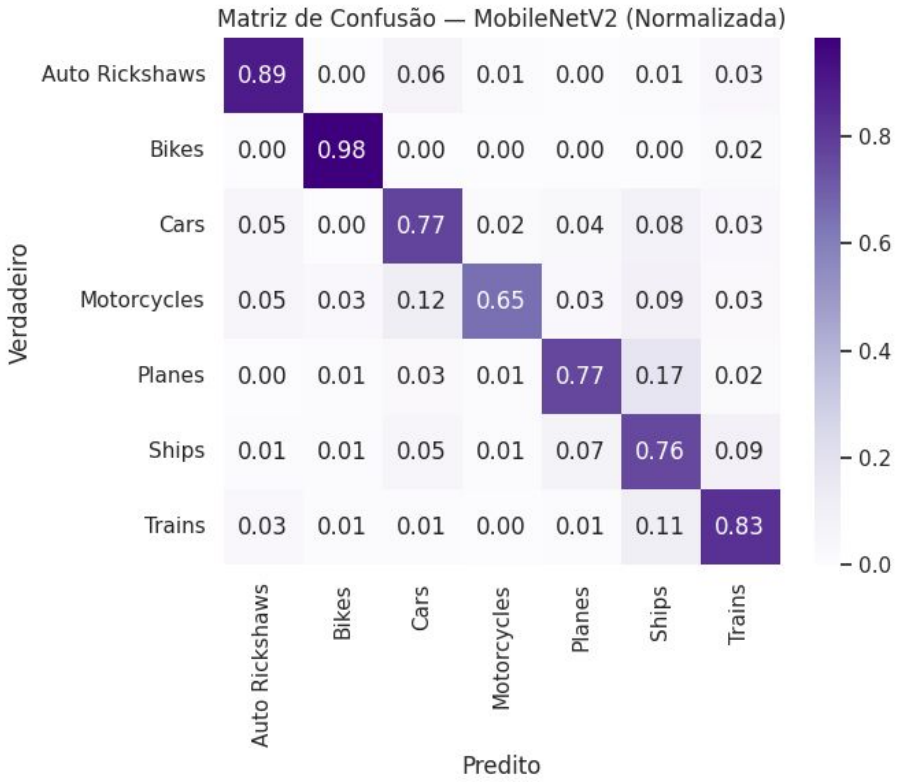
Foram utilizadas três arquiteturas principais: **MobileNetV2** e **EfficientNetB0** (com **transfer learning**) e uma **SmallCNN** personalizada treinada do zero.

Nos modelos de transfer learning, foi adotada a estratégia de **congelar as camadas convolucionais pré-treinadas** inicialmente, treinando 10 épocas apenas com as camadas finais de classificação. Em seguida, realizou-se um **fine-tuning** por mais 10 épocas, descongelando parcialmente (as últimas 30 camadas) para ajustar os pesos às características do dataset.

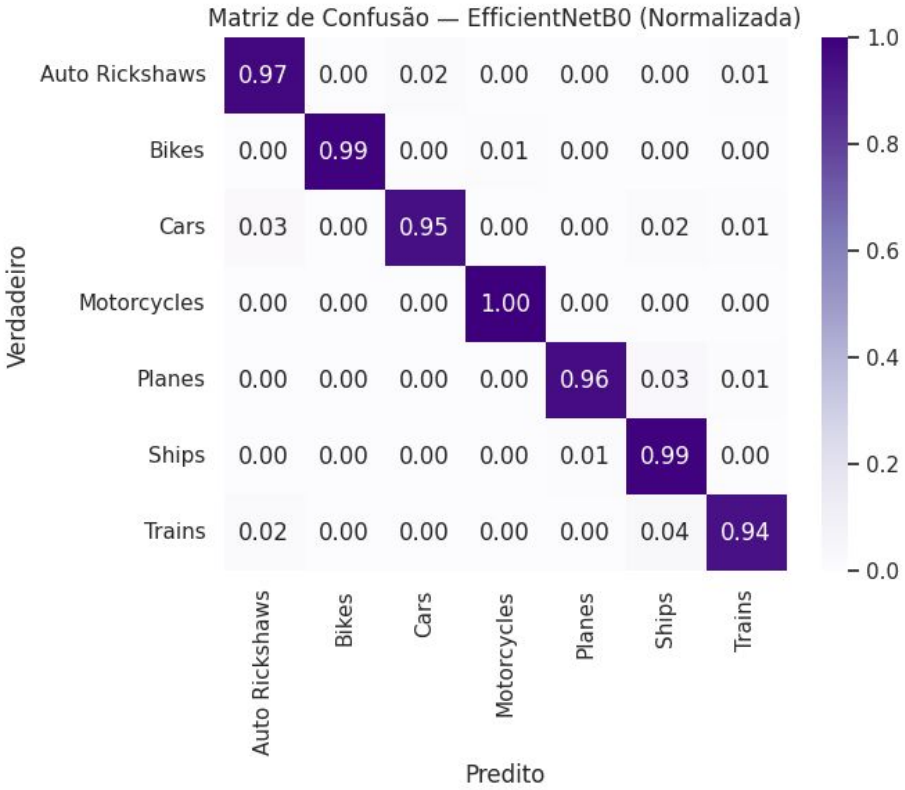
Os modelos foram treinados por cerca de **20 a 25 épocas no total**, com **batch size 32**, utilizando **Early Stopping** e **ReduceLROnPlateau** para evitar overfitting e otimizar o aprendizado.

Todo o processo foi implementado em **TensorFlow/Keras**, com uso de **GPU no Google Colab** para acelerar o treinamento.

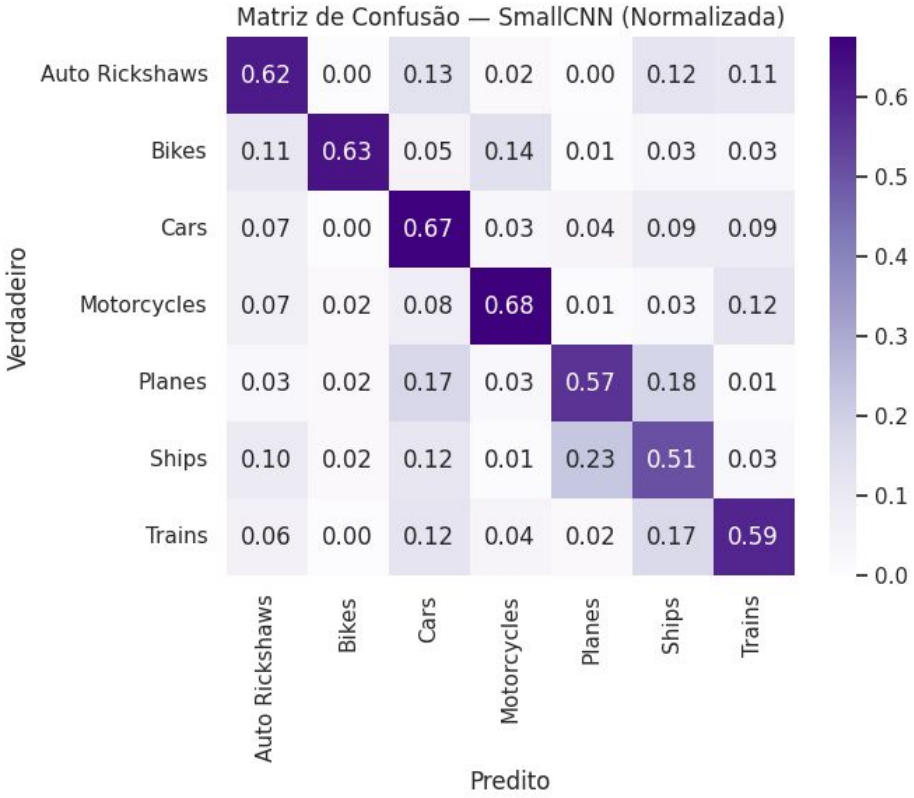
Resultados Quantitativos



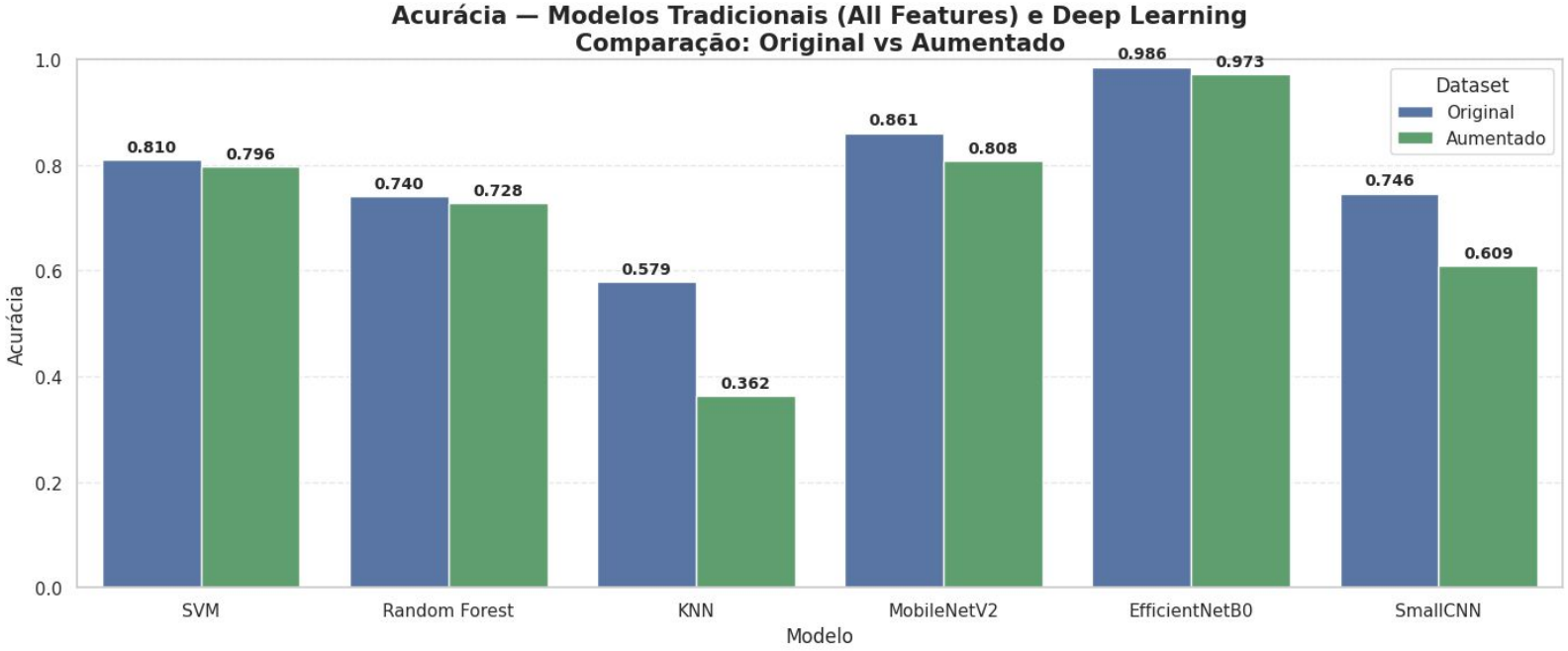
Resultados Quantitativos



Resultados Quantitativos



Resultados Quantitativos

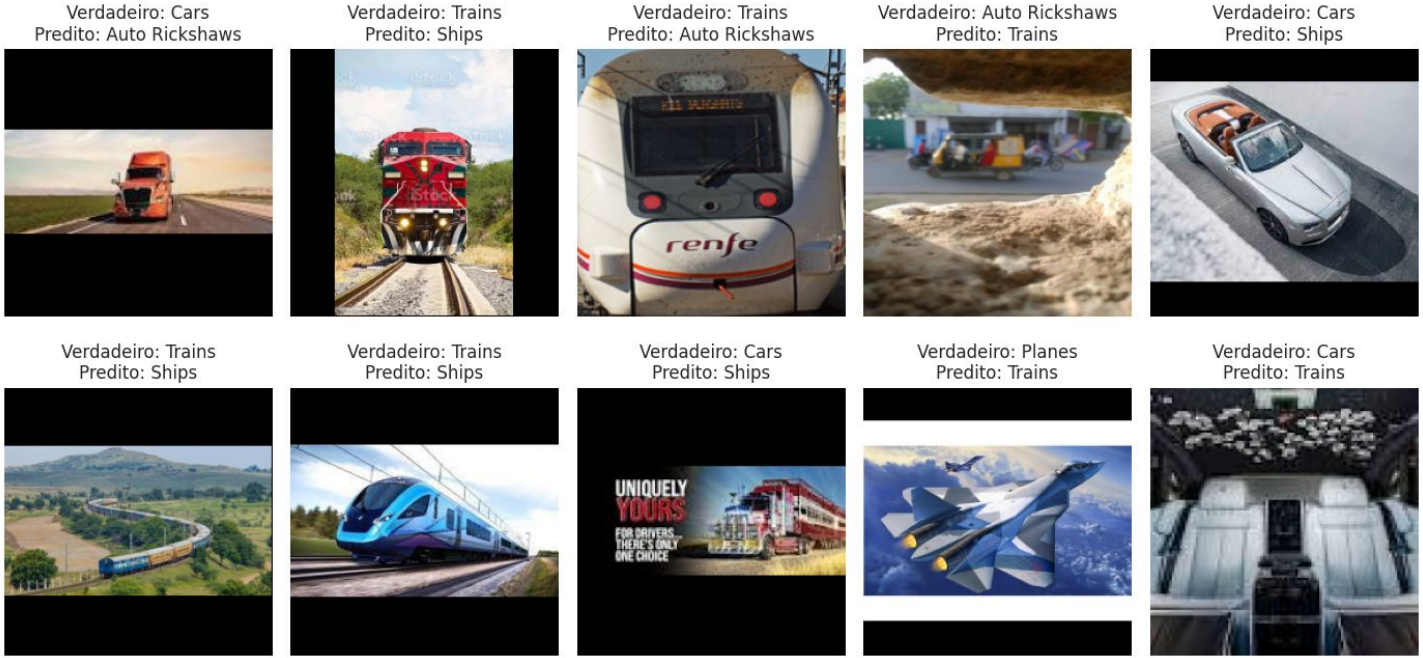


Resultados Qualitativos

Acertos EfficientNet

Resultados Qualitativos

Erros EfficientNet



Ablation Study

Arquiteturas e camadas congeladas (Transfer Learning):
Testamos **MobileNetV2** e **EfficientNetB0** com *transfer learning*, congelando inicialmente toda a base pré-treinada e depois liberando apenas as últimas camadas para *fine-tuning*.

Conclusão: **MobileNetV2 se beneficiou do fine-tuning**, enquanto EfficientNetB0 continuou apresentando baixo desempenho.

Hiperparâmetros:
Learning rate (1e-3 ⟶ 1e-5 no fine-tuning)

Batch size = 32

Early stopping + *ReduceLROnPlateau* evitaram overfitting e ajudaram na convergência.

Conclusões

Principais conclusões

A abordagem de **Deep Learning superou consistentemente os métodos tradicionais**, sobretudo com arquiteturas pré-treinadas.

O **EfficientNetB0 treinado no dataset original foi o melhor modelo**, alcançando **acurácia de 98,69%**, mostrando excelente capacidade de generalização.

O data augmentation clássico **não melhorou a performance**; na verdade, provocou **pequena queda** nos modelos, indicando que o dataset original já possuía variabilidade suficiente.

Possíveis Melhorias Futuras

Arquiteturas Mais Avançadas

EfficientNetV2 – mais rápida e com melhor desempenho que B0.

ConvNeXt – arquitetura moderna inspirada em Transformers.

Vision Transformers (ViT) – capturam conexões de alto nível entre pixels, ampliando a capacidade de generalização do modelo.

Treinamento Mais Otimizado

Fine-tuning mais profundo
Liberar um número maior de camadas para permitir ajuste fino completo.

Schedulers avançados
Cosine Annealing: reduz a LR suavemente ao longo do treino.
One-Cycle Policy: aumenta e depois decresce a LR para acelerar a convergência.